Katedra informatiky
Přírodovědecká fakulta
Univerzita Palackého v Olomouci

BAKALÁŘSKÁ PRÁCE

Rozšíriteľná knižnica pre správu Docker kontajnerov implementovaná v Go

2025

Samuel Táborský

Vedoucí práce:
Mgr. Tomáš Urbanec, Ph.D.

Studijní program: Informatika,
Specializace: Programování a vývoj software

Bibliografické údaje

Autor:	Samuel Táborský
Název práce:	Rozšíriteľná knižnica pre správu Docker kontajnerov implementovaná v Go
Typ práce:	bakalářská práce
Pracoviště:	Katedra informatiky, Přírodovědecká fakulta, Univerzita Palackého v Olomouci
Rok obhajoby:	2025
Studijní program:	Informatika, Specializace: Programování a vývoj software
Vedoucí práce:	Mgr. Tomáš Urbanec, Ph.D.
Počet stran:	12
Přílohy:	elektronická data v úložišti katedry informatiky
Jazyk práce:	slovenský

Bibliographic info

Author:	Samuel Táborský
Title:	An extensible library for Docker Container management implemented in Go
Thesis type:	bachelor thesis
Department:	Department of Computer Science, Faculty of Science, Palacký University Olomouc
Year of defense:	2025
Study program:	Computer Science, Specialization: Programming and Software Development
Supervisor:	Mgr. Tomáš Urbanec, Ph.D.
Page count:	12
Supplements:	electronic data in the storage of department of computer science
Thesis language:	Slovak

Anotácia

Táto bakalárska práca sa zaoberá návrhom a implementáciou knižnice v jazyku Go na správu Docker kontajnerov. Opisuje teoretické východiská kontajnerizácie a prístup cez Docker API, pričom sa sústreďuje na modulárnu architektúru umožňujúcu rozšíriteľnosť aj podporu rôznych kontajnerových runtime. Praktická časť práce zahŕňa implementáciu knižnice a sprievodnej ukážkovej aplikácie pre správu kontajnerov, sledovanie udalostí a monitoring prostriedkov. Výsledné riešenie je overené testami a predstavuje základ pre ďalší vývoj a rozšírenie v open-source prostredí.

Synopsis

This bachelor's thesis focuses on the design and implementation of a Go-based library for Docker container management. It outlines the fundamentals of containerization and Docker API access, with emphasis on a modular architecture enabling extensibility and support for multiple container runtimes. The practical part includes the implementation of the library and an accompanying demo application for container management, event monitoring, and resource tracking. The solution is validated by testing and provides a foundation for further development and open-source extension.

Kľúčové slová: Docker; kontajnerizácia, knižnica; jazyk Go

Keywords: Docker; containerization; library; Go language

Poďakovania

Odevzdaním tohto textu jeho autor/ka cestne vyhlasuje, že celú prácu vrátane príloh vypracoval/a samostatne a za použitia iba zdrojov spomínaných v texte práce a uvedených v zozname literatúry.

Obsah

1 Úvod **6**
1.1 Motivácia a ciele práce 6
1.2 Vymedzenie problému 6
1.3 Prehľad existujúcich riešení 6
1.4 Prínos a očakávania 6

2 Teoretické základy **6**
2.1 Virtualizácia a kontajnerizácia 6
2.2 Docker a jeho architektúra 6
2.3 Programovací jazyk Go a jeho ekosystém 6
2.4 Softvérové architektúry a rozšíriteľnosť knižníc 7

3 Analýza problému **7**
3.1 Požiadavky na systém 7
3.2 Prehľad dostupných knižníc a nástrojov 7
3.3 Návrh architektúry riešenia 7

4 Návrh a implementácia riešenia **7**
4.1 Návrh knižnice 7
4.2 Implementácia Docker klienta 7
4.3 Testovanie a validácia 8
4.4 Implementácia ukážkovej aplikácie 8

5 Možné budúce rozšírenia **8**
5.1 Podpora iných runtimes (Podman, containerd! 8
5.2 Správa zväzkov a sieťovanie 8
5.3 Open-source pokračovanie 8

Záver **9**

Conclusions **10**

6 Obsah elektronických dát **10**

Zoznam skratiek **12**

1 Úvod

1.1 Motivácia a ciele práce

V krátkosti predstavím moju prácu spolu s motiváciou, vymedzím si ciele, ktoré chcem aby práca spĺňala.

1.2 Vymedzenie problému

Komplexnejšie predstavím projekt a vysvetlím obmedzenia a rozsah môjho riešenia, ako aj dôvody na tieto rozhodnutia.

1.3 Prehľad existujúcich riešení

Popíšem, aké iné nástroje a aplikácie existujú, aké sú ich obmedzenia a výhody.

1.4 Prínos a očakávania

Prednesiem moje očakávania od hotového projektu a jeho prínosy, ako napríklad možnosť pokračovať vo vývoji ako súčasť open-source komunity.

2 Teoretické základy

2.1 Virtualizácia a kontajnerizácia

V tejto kapitole popíšem históriu kontajnerizácie a jej vzniku. Zameriam sa takisto na porovnanie s bežnou virtualizáciou, prednesiem výhody a nevýhody oboch prístupov, ako aj moderné použitie kontajnerov v cloudových službách.

2.2 Docker a jeho architektúra

V tejto kapitole predstavím Docker a Docker Engine, jeho architektúru, históriu a komponenty. Popíšem ako vyzerá REST API, s ktorým bude moja knižnica komunikovať, a prednesiem technické možnosti, ktoré Docker poskytuje na správu kontajnerov.

2.3 Programovací jazyk Go a jeho ekosystém

Predstavím jazyk Go ako moderné pokračovanie nízkoúrovňových jazykov typu C, jeho vlastnosti a špecifické možnosti. Popíšem štruktúru programov napísaných v jazyku Go, používanie modulov a správne praktiky pre písanie efektívneho, znovupoužiteľného a rozšíriteľného kódu.

2.4 Softvérové architektúry a rozšíriteľnosť knižníc

V tejto kapitole popíšem rôzne architektúry softvéru, ich výhody a nevýhody. Rovnako sa zameriam aj na praktiky, ktoré umožňujú zjednodušiť vývoj a následné udržovanie a rozširovanie softvérových aplikácií. Skrátka predstavím aj spôsob, ktorým docielim rozšíriteľnosť mojej knižnice o podporu ďalších backendov, ako je napríklad Podman.

3 Analýza problému

3.1 Požiadavky na systém

V tejto podkapitole vymedzím požiadavky na funkčnosť aplikácie (ako napríklad možnosti správy kontajnerov, monitorovanie ich stavu atď.), ako aj požiadavky na štruktúru aplikácie, jej rozšíriteľnosť a čistotu kódu.

3.2 Prehľad dostupných knižníc a nástrojov

Predstavím všetky nástroje a knižnice, ktoré bude moja aplikácia využívať, na čo ich bude využívať a prečo niektoré časti nebudem písať ja. Ukážem rozdiely medzi REST API a SDK, a takisto v krátkosti predstavím nástroje a knižnice, ktoré by mohli byť použité pre rozšírenie na iné kontajnerové runtimes.

3.3 Návrh architektúry riešenia

Popíšem architektúru celej aplikácie, jej rozdelenie na oddelené vrstvy a dôvody na tieto rozhodnutia. Predstavím základné rozhranie aplikácie, ktoré bude spoločné pre všetky runtimes. Rovnako popíšem dizajn, ktorý mi umožňuje v budúcnosti aplikáciu rozšíriť.

4 Návrh a implementácia riešenia

4.1 Návrh knižnice

Popíšem štruktúru knižnice a repozitára, rozdelenie na samostatné časti, ďalej predstavím typy a verejné rozhranie (API). Vysvetlím moje použitie submodulov namiesto reexportu v hlavnom module, ako aj možnosť prejsť v budúcnosti na tento spôsob.

4.2 Implementácia Docker klienta

Opíšem implementáciu klienta, ktorý komunikuje s Docker Engine cez REST API a SDK. Ukážem funkcionalitu, ktorú klient umožňuje (správa životného cyklu, monitorovanie, sledovanie udalostí - eventov, správa kontajnerov).

4.3 Testovanie a validácia

Ukážem, akým spôsobom je aplikácia testovaná a ktoré jej časti sú pokryté. Použijem unit testy a integračné testy.

4.4 Implementácia ukážkovej aplikácie

Predstavím jednoduchú ukážkovú aplikáciu, ktorá demonštruje spôsob a možnosti použitia tejto knižnice.

5 Možné budúce rozšírenia

5.1 Podpora iných runtimes (Podman, containerd!

5.2 Správa zväzkov a sieťovanie

5.3 Open-source pokračovanie

Záver

Závěr práce v „českém“ jazyce.

Conclusions

Thesis conclusions in “English”.

6 Obsah elektronických dát

Na samotném konci textu práce je uveden stručný popis obsahu elektronických dat odevzdaných v systému katedry informatiky spolu s textem. Tato data jsou nedílnou součástí práce a tvoří (datovou) přílohu textu práce. Povinné položky struktury dat jsou:

`text/`
: Adresář s textem práce ve formátu PDF, vytvořený s použitím závazného stylu KI PřF UP v Olomouci pro závěrečné práce, včetně všech (textových) příloh, a všechny soubory potřebné pro bezproblémové vytvoření PDF dokumentu textu (případně v ZIP archivu), tj. zdrojový text textu a příloh, vložené obrázky, apod.

`README.*`
: Textový soubor (s příponou např. `.txt`) s informacemi o opakovatelném způsobu použití ostatních dat práce – typicky plně reprodukovatelný co nejúplnější funkční postup zprovoznění software vytvořeného v rámci práce, tzn. jeho případné instalace/nasazení a spuštění, včetně uvedení všech požadavků pro bezproblémový provoz; za zprovoznění software se nepovažuje zpřístupnění (např. po Internetu) již někde zprovozněného software.

`*`
: Adresáře a soubory s veškerými ostatními autorskými daty práce (případně v ZIP archivu) – typicky spustitelné a další soubory software vytvořeného v rámci práce potřebné pro bezproblémový provoz software, případně jeho instalační program, a kompletní zdrojové texty software a další data nutná pro plně reprodukovatelné korektní vytvoření spustitelných souborů.

Dále mohou data obsahovat například:

- ukázková a testovací data použitá v práci nebo pro potřeby posouzení práce v rámci její obhajoby,
- položky bibliografie v elektronické podobě, příp. jiná relevantní literatura a dokumentace vztahující se k práci,
- cizí data (software) potřebná pro bezproblémové použití autorských dat práce (software), která nejsou standardní součástí předpokládaného (softwarového) vybavení uživatele.

U veškerých cizích obsažených materiálů jejich zahrnutí dovolují podmínky pro jejich veřejné šíření nebo přiložený souhlas držitele práv k užití. Pro všechny použité (a citované) materiály, u kterých toto není splněno a nejsou tak obsaženy, je uveden jejich zdroj, např. webová adresa, v bibliografii nebo textu práce nebo souboru `README.*`.